Institut Teknologi Bandung

Directorate of Continuing Professional Education

in partnership with

CHAPTER 5

Fundamentals of Machine Learning

The tension between optimisation and generalisation — why it is unavoidable, how to measure it, and which remedies actually help.

Duration 3 hours (2 sessions) **Instructor** Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 5

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. State the **optimisation vs generalisation** tension, and name the three causes of overfitting: noisy data, ambiguous features, and rare features.
2. Explain the **manifold hypothesis**, and why generalisation in deep learning is *interpolation* rather than reasoning.
3. Choose an evaluation protocol — **hold-out, K-fold, iterated K-fold** — and avoid the three pitfalls that invalidate it.
4. Set a **common-sense baseline** before training anything.
5. Diagnose the three training failures: **won't start, won't generalise, won't overfit** — and apply the right remedy to each.
6. Apply **dataset curation, feature engineering, early stopping, capacity reduction, weight regularisation, and dropout**, and know when each fits.

BOOK
Chapter 5 — full text

NOTEBOOK
01_spurious_correlations.ipynb

NOTEBOOK
02_shuffled_labels_and_manifolds.ipynb

NOTEBOOK
03_evaluation_protocols.ipynb

NOTEBOOK
04_model_capacity.ipynb

NOTEBOOK
05_regularisation_l2_dropout.ipynb

REPO
Author's official notebook

ROADMAP

From "it runs" to "it may be released"

Chapter 4 introduced overfitting as an event. Chapter 5 turns it into **a way of thinking**, and nearly every best practice in the rest of the book manages the same single tension.

5.1 · Generalisation
What causes overfitting, and why deep learning generalises at all.

5.2 · Evaluation
Three protocols, a common-sense baseline, and three pitfalls.

5.3 · Improving fit
Three training failures and their remedies. The goal: be **able** to overfit.

5.4 · Improving generalisation
Curation, features, early stopping, and three kinds of regularisation.

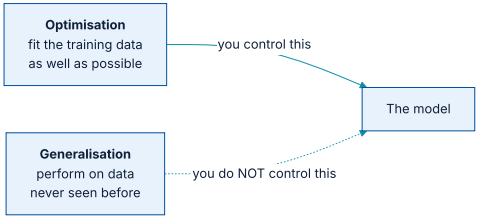
01

Generalisation: the goal

Why overfitting happens in every problem, without exception.

SECTION 5.1

The one tension everything else serves



You can only act on one side of this picture.

> *The goal of the game is to get good generalisation, of course, but you don't control generalisation; you can only fit the model to its training data. If you do that too well, overfitting kicks in and generalisation suffers.*
>
> Chollet & Watson, section 5.1

SECTION 5.1.1

The canonical curve — and it is universal

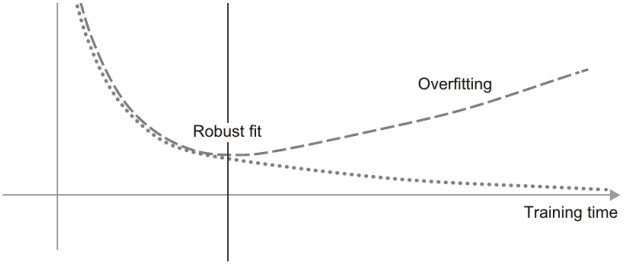


Figure 5.1 — the same shape appears with every model type and every dataset. — Chollet & Watson, *Deep Learning with Python*, 3rd ed. (Manning)

- **Underfit** — progress still available; the network has not yet modelled the relevant patterns.
- **Robust fit** — the best point. Narrow, and visible only through validation metrics.
- **Overfit** — the model starts learning patterns **specific to the training data** that mislead on new data.

SECTION 5.1.1

Cause 1 — noisy training data

A model that goes out of its way to accommodate those outliers gets worse on everything near them: a 4 resembling a mislabelled 4 **ends up classified as a 9**

Cause 2 — ambiguous features

No objective boundary. Is this banana unripe, ripe, or rotten? Different human labellers disagree on the same photograph.

Genuine randomness. The same atmospheric pressure reading is sometimes followed by rain and sometimes by clear sky.

A model overfits such data by being **too confident** in the ambiguous region. A robust fit **ignores individual points and looks at the bigger picture**

SECTION 5.1.1

Cause 3 — rare features and spurious correlations

The machine version: the word *cherimoya* appears in exactly one training review, which happens to be negative. A poorly regularised model gives it enormous weight and thereafter **condemns every text that mentions the fruit**

> And it need not be that rare. A word appearing in **100** samples, positive **54%** of the time, may be pure statistical fluke — and the model will use it anyway. Chollet calls this **one of the most common sources of overfitting**.

Proving it: add pure noise, lose accuracy

listing 5.1 — two comparison sets PYTHON

```python
(train_images, train_labels), _ = mnist.load_data()
train_images = train_images.reshape((60000, 28 * 28)).astype("float32") / 255

train_images_with_noise_channels = np.concatenate(
    [train_images, np.random.random((len(train_images), 784))], axis=1)

train_images_with_zeros_channels = np.concatenate(
    [train_images, np.zeros((len(train_images), 784))], axis=1)
```

A human classifier would be entirely unaffected by either transformation.

. . . and the model is not unaffected

listing 5.2 — same model, two datasets PYTHON

```python
model = get_model()
history_noise = model.fit(train_images_with_noise_channels, train_labels,
                          epochs=10, batch_size=128, validation_split=0.2)

model = get_model()
history_zeros = model.fit(train_images_with_zeros_channels, train_labels,
                          epochs=10, batch_size=128, validation_split=0.2)
```

−1 point

validation accuracy, purely from spurious correlations

One percentage point, from information carrying **no signal at all**. Add more noise channels and it degrades further. Hence **feature selection**: score each feature's usefulness — mutual information with the labels, say — and **keep only what clears a threshold**

A model will fit absolutely anything

listing 5.4 — randomly shuffled labels PYTHON

```python
random_train_labels = train_labels[:]        # copy
np.random.shuffle(random_train_labels)       # destroy every input-target relation

model.fit(train_images, random_train_labels,
          epochs=100, batch_size=128, validation_split=0.2)
```

OUTPUT

```
training loss      : falls steadily, smoothly
validation accuracy: stuck at ~10%  (= the random baseline for 10 classes)
```

The training loss falls even though **nothing is learnable**. The model is **memorising, like a Python dictionary**. So the ability to fit is no evidence that the problem is solvable.

SECTION 5.1.2

So why does it generalise at all?

MNIST inputs are 28×28 arrays of bytes, so there are $\mathbf{256}^{784}$ possible inputs — **more than there are atoms in the universe**. Almost none of them look like handwriting.

SECTION 5.1.2

The manifold hypothesis

> *The manifold hypothesis posits that all natural data lies on a low-dimensional manifold within the high-dimensional space where it is encoded.*
>
> Chollet & Watson, section 5.1.2

- A **manifold** is a lower-dimensional subspace that is locally similar to a linear space — a smooth curve is a 1D manifold inside a 2D plane.
- The subspace of valid digits is **continuous**: perturb a sample slightly and it is still the same digit.
- Any two digits are joined by a **smooth path** of intermediate images that all still look like digits.
- It holds for handwriting, human faces, tree morphology, the human voice, and **even natural language**.

SECTION 5.1.2

Interpolation, and what it is not

What the model can do

- Make sense of an unseen point by relating it to nearby points on the manifold.
- Understand the whole space from a sample of it — *filling in the blanks*.
- Chollet calls this **local generalisation**.

What it cannot

- **Extreme generalisation**, which humans do constantly.
- You can spend a week in New York, a week in Shanghai, and a week in Bangalore **without thousands of lifetimes of rehearsal**.
- That rests on abstraction, symbolic models, reasoning, and innate priors — what we call **reason**, not intuition.

Interpolation is only **the tip of the iceberg**. Treating it as the whole of generalisation is a mistake; chapter 19 returns to this.

SECTION 5.1.2

And it is not linear interpolation either

The pixel-wise average of two MNIST digits **is usually not a valid digit at all**. Every point on the latent manifold is; the straight line between two points in pixel space is not.

SECTION 5.1.2

Why deep learning is suited to this

1. The curve has enough parameters to fit **anything**; trained long enough it simply memorises.
2. But the data is **not** scattered points — it forms a structured, low-dimensional manifold.
3. Because the fitting happens **gradually and smoothly**, there is an intermediate moment when the model curve approximates the data's own manifold. That is the robust fit.

SECTION 5.1.2

Two properties that make it work

A smooth, continuous mapping

Required, because the model must be differentiable — and that smoothness is exactly what helps it approximate a manifold, which has the same property.

Architecture priors

Models are structured to mirror the *shape* of the information in their data — especially image models (ch. 8–12) and sequence models (ch. 13).

SECTION 5.1.3

Training data is paramount

> *The only thing you will find in a deep learning model is what you put into it: the priors encoded in its architecture and the data it was trained on.*
>
> Chollet & Watson, section 5.1.3

- The power to generalise is more a consequence of **your data's natural structure** than of any property of your model.
- Curve fitting needs a **dense sampling** of the input space — especially near decision boundaries.
- Sparse sampling → the learned curve does not match the latent space, and **interpolation goes wrong**.

Hence the single most effective move available to you: **train on more data, or on better data.**

SECTION 5.1.3

When more data is not an option

> If a network can only afford to memorise a few patterns, or only very regular ones, optimisation **forces it towards the most prominent patterns** — the ones with a better chance of generalising.

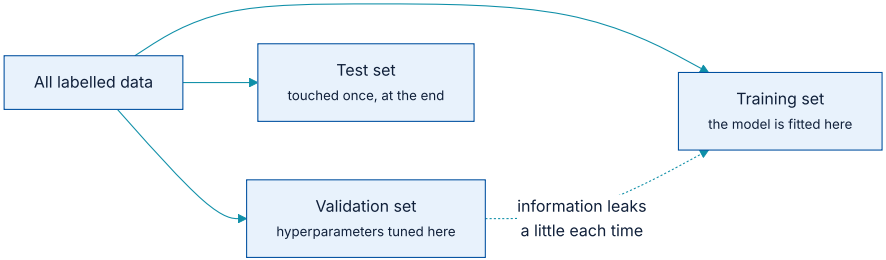
Fighting overfitting this way is called **regularisation**, and section 5.4.4 covers it in full.

02

Evaluating machine learning models

You can only control what you can observe.

Why two sets are not enough



Tuning on the validation set is itself a form of learning — and it leaks.

Hyperparameters (layer count, layer size) are what you tune; **parameters** (the weights) are what the model learns. Tuning is a search, and searches can overfit.

SECTION 5.2.1

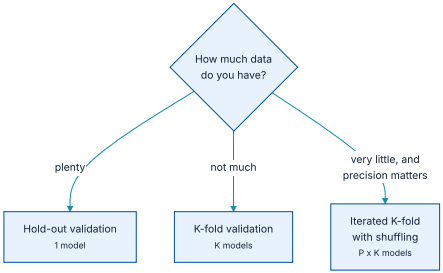
Information leaks, one bit at a time

- Do it **once**, for one parameter: a few bits leak, and the set stays trustworthy.
- Do it **many times** — run, evaluate, adjust, repeat — and an increasingly significant amount leaks.
- You end up with a model that does artificially well on validation, **because that is what you optimised it for**.

> Hence the third set. The model must have had **no access to any information about the test set, even indirectly**. If anything at all was tuned on test performance, your measure of generalisation is already flawed.

SECTION 5.2.1

Three evaluation protocols



Pick one. In most cases the first is good enough.

The tell that hold-out has failed you: **different random shuffles before splitting give very different scores**. That is the signal to move to K-fold.

K-fold, and the line people get wrong

listing 5.6 — K-fold cross-validation PYTHON

```python
k = 3
num_validation_samples = len(data) // k
np.random.shuffle(data)
validation_scores = []

for fold in range(k):
    validation_data = data[num_validation_samples * fold :
                           num_validation_samples * (fold + 1)]
    training_data = np.concatenate(
        [data[: num_validation_samples * fold],
         data[num_validation_samples * (fold + 1) :]])

    model = get_model()            # a BRAND-NEW, untrained model every fold
    model.fit(training_data, ...)
    validation_scores.append(model.evaluate(validation_data, ...))

validation_score = np.average(validation_scores)
```

Reusing an already-trained model across folds carries knowledge between them and **invalidates the entire estimate**

SECTION 5.2.1

And then the final model

after the folds — PYTHON

```python
model = get_model()
model.fit(data, ...)                          # all non-test data
test_score = model.evaluate(test_data, ...)   # touched once, here
```

Iterated K-fold with shuffling repeats the whole procedure P times with a fresh shuffle each time. It trains P × K models — expensive, and the book notes it is especially useful in Kaggle competitions.

SECTION 5.2.2

The altimeter on an invisible rocket

> *Training a deep learning model is a bit like pressing a button that launches a rocket in a parallel world. You can't hear it or see it … The only feedback you have is your validation metrics — like an altitude meter on your invisible rocket.*
>
> Chollet & Watson, section 5.2.2

Which raises the question the next slide answers: **what altitude did you start at?**

SECTION 5.2.2

Set the baseline before you train anything

PROBLEM	COMMON-SENSE BASELINE	WHY
MNIST	**> 0.10**	A random classifier over 10 balanced classes.
IMDB	**> 0.50**	Two classes, balanced 50:50.
Reuters	≈ **0.18–0.19**	46 classes, but very unevenly distributed.
Binary, 90:10 split	**> 0.90**	A classifier that always answers A already scores 0.90. You must beat that.

If you **cannot beat a trivial solution, your model is worthless**. Either the model is wrong, or — and this happens — **the problem cannot be approached with machine learning at all**

SECTION 5.2.3

Three pitfalls that invalidate an evaluation

1 · Data representativeness

Samples ordered by class, then the first 80% taken as training: you train on classes 0–7 and test on 8–9. *A ridiculous mistake, and surprisingly common.* **Shuffle before splitting.**

2 · The arrow of time

Predicting the future from the past? **Do not shuffle.** It creates a **temporal leak**: the model trains on data from the future. All test data must be later than all training data.

3 · Redundancy

Duplicate points are common in real data. Shuffle and split, and copies land in both training and validation — you are testing on your training data. **The worst thing you can do.**

SECTION 5.2.3

Why transactional data hits two of them at once

> Sequences of events are **temporal**, and the same subject recurs across many rows. Splitting randomly by row **violates pitfalls 2 and 3 in a single step**

The fix is to split by **time** and by **subject**, not by row — and to state which you did when you report a score.

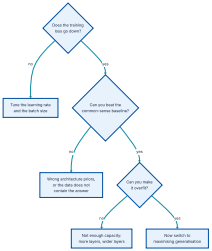
03

Improving model fit

To reach the perfect fit, you must first overfit.

SECTION 5.3

Diagnose before you treat



Three questions, asked in order. Each failure has a different remedy.

> *To achieve the perfect fit, you must first overfit. Since you don't know in advance where the boundary lies, you must cross it to find it.*
>
> Chollet & Watson, section 5.3

Failure 1 — training never starts

listing 5.7 — too high PYTHON

```python
model.compile(
    optimizer=keras.optimizers.RMSprop(
        learning_rate=1.0),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

OUTPUT

```
accuracy stalls at 20%-40%
```

listing 5.8 — reasonable PYTHON

```python
model.compile(
    optimizer=keras.optimizers.RMSprop(
        learning_rate=1e-2),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

OUTPUT

```
the model now trains
```

All these parameters interact, so the advice is narrow: **tune the learning rate and the batch size, and hold the rest constant.** A bigger batch gives **less noisy, more informative gradients**

SECTION 5.3.2

Failure 2 — it trains but does not generalise

The book calls this **the worst situation you can find yourself in**: it signals something fundamentally wrong, and **it is often not easy to tell what**

The data may not contain the answer

Exactly what happened with the shuffled MNIST labels: it trains fine, and validation sits at 10% forever.

The architecture priors may be wrong

Chapter 13 shows a timeseries problem where a densely connected model cannot beat a trivial baseline and a recurrent one generalises well.

Practical advice: **read up on architecture best practices for your kind of task** — you are almost certainly not the first to attempt it.

Failure 3 — it cannot be made to overfit

listing 5.9 — insufficient capacity

PYTHON

```python
model = keras.Sequential([layers.Dense(10, activation="softmax")])
model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
history_small = model.fit(train_images, train_labels,
                          epochs=20, batch_size=128, validation_split=0.2)
```

OUTPUT

```
validation loss falls to 0.26 and simply stays there
```

You can fit, but you cannot clearly overfit — even after many passes. **It should always be possible to overfit**, so this points at capacity.

SECTION 5.3.3

Too little, right, and too much

MODEL	ARCHITECTURE	WHAT HAPPENS	FIGURE
Under-capacity	`Dense(10, softmax)` alone	Validation loss reaches 0.26 and stops. Fits, but **cannot clearly overfit**.	5.14
Right capacity	2 × `Dense(128, relu)`	Fits fast, starts overfitting **after eight epochs**. Exactly the shape you want.	5.15
Over-capacity	3 × `Dense(2048, relu)`	Overfits **immediately**; training loss near zero very quickly, validation loss noisy.	5.16

The workflow: **start with few layers and parameters, and grow until validation loss stops improving.** There is no formula for the right size — it is found on the validation set, **never on the test set**

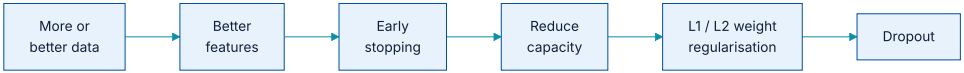
04

Improving generalisation

Five moves, in order of how much they are worth.

SECTION 5.4

The order to try things in



Left to right, cheapest and most effective first.

Every one of these is downstream of the same idea: make the model's curve smoother, or make interpolation easier.

SECTION 5.4.1

Dataset curation pays the most, and gets skipped the most

> *Deep learning is curve fitting, not magic.*
>
> Chollet & Watson, section 5.4.1

1. **Make sure you have enough data.** You need a dense sampling of the input-cross-output space. Problems that look impossible sometimes become solvable with a larger dataset.
2. **Minimise labelling errors.** Look at your inputs for anomalies; proofread the labels.
3. **Clean the data and handle missing values.** Chapter 6 covers how.
4. **Do feature selection** if you have many features and are unsure which are useful.

And know the limit: if the problem is **overly noisy or fundamentally discrete** — sorting a list, say — deep learning **will not help you**, at any data volume.

SECTION 5.4.2

Feature engineering: the clock face

LEVEL	WHAT GOES IN	WHAT IT TAKES TO SOLVE
Raw data	The pixel grid of a clock	A convolutional network, and considerable compute.
Better features	(x, y) coordinates of each hand's tip	A simple machine learning algorithm suffices.
Better still	The angle θ of each hand (polar coordinates)	**No machine learning at all** — rounding and a dictionary lookup.

That is the essence of it: **making a problem easier by expressing it more simply**. Make the latent manifold smoother and better organised. It requires understanding the problem deeply.

SECTION 5.4.2

Deep learning removed the need — but not entirely

Still worth it — reason 1

Good features solve problems **with fewer resources**. Using a ConvNet to read a clock face would be absurd.

Still worth it — reason 2

Good features solve problems **with far less data**. A model's ability to find its own features depends on having a lot of it.

SECTION 5.4.3

Early stopping

The chapter-4 way — train longer than needed to find the best epoch, then train a fresh model for exactly that many.

> Standard, but it means **doing the work twice**

The better way — save the model at the end of each epoch and keep the best. In Keras this is the `EarlyStopping` callback, which halts as soon as validation stops improving **while remembering the best state**.

> Callbacks are covered in **chapter 7**.

Regularisation 1 — make the model smaller

VERSION	HIDDEN LAYERS	STARTS OVERFITTING	BEHAVIOUR
Reference	2 × `Dense(16)`	epoch 4	The comparison point.
Smaller	2 × `Dense(4)`	epoch 6	Overfits later, and **degrades more slowly** once it does.
Much larger	2 × `Dense(512)`	epoch 1	Overfits almost immediately and far more severely; validation loss noisier, training loss near zero fast.

Regularisation 2 — constrain the weights

listing 5.13–5.14 — weight regularisers PYTHON

```python
from keras.regularizers import l2
from keras import regularizers

model = keras.Sequential([
    layers.Dense(16, kernel_regularizer=l2(0.002), activation="relu"),
    layers.Dense(16, kernel_regularizer=l2(0.002), activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])

regularizers.l1(0.001)                    # L1
regularizers.l1_l2(l1=0.001, l2=0.001)    # both at once
```

`l2(0.002)` adds `0.002 * weight ** 2` per coefficient to the loss. The penalty applies **only during training**, so **training loss will read much higher than test loss** — expected, not a bug.

SECTION 5.4.4

L1 or L2, and when weight regularisation stops helping

KIND	PENALTY ADDED TO THE LOSS	EFFECT
L1	Proportional to the **absolute value** of the weights.	Pushes weights towards **sparsity** — many exactly zero.
L2	Proportional to the **square** of the weights.	Pushes all weights small. Also called **weight decay** — different name, identical mathematics.

Weight regularisation is **typically used on smaller models**. Large models are so overparameterised that constraining weight values **has little effect on capacity**. For those, dropout is preferred.

Regularisation 3 — dropout

Figure 5.21 — the rescaling is done during training so that test time needs no adjustment at all.

The **dropout rate** is the fraction zeroed out, usually between **0.2 and 0.5**.

SECTION 5.4.4

. . . and how it looks in a model

listing 5.15 — dropout on the IMDB model PYTHON

```python
model = keras.Sequential([
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(16, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(1, activation="sigmoid"),
])
```

On the IMDB model this is a clear improvement over the reference — and it reaches a **lower minimum validation loss than L2 did**.

SECTION 5.4.4

Where dropout came from

> *I went to my bank. The tellers kept changing and I asked one of them why. He said he didn't know but they got moved around a lot. I figured it must be because it would require cooperation between employees to successfully defraud the bank. This made me realize that randomly removing a different subset of neurons on each example would prevent conspiracies and thus reduce overfitting.*
>
> Geoff Hinton, quoted in Chollet & Watson, section 5.4.4

The mechanism in one line: injecting noise into a layer's outputs **breaks up accidental patterns** — Hinton's *conspiracies* — that the model would otherwise start memorising.

SECTION 5.4.4

Which technique, when

TECHNIQUE	BEST SUITED TO	NOTES
More / better data	Always, where possible.	The largest return. Adding overly noisy data harms instead.
Better features	Small data; well-understood problems.	Can remove the need for a large model entirely.
Reduce capacity	Small models; small datasets.	Find the compromise — do not tip into underfitting.
L1 / L2	**Smaller** deep learning models.	On very large models, constraining weights **does little**
Dropout	**Large** models, where weight decay is weak.	Rates of 0.2–0.5. Beat L2 on the IMDB benchmark.

One condition governs all of them: **regularisation must be guided by an accurate evaluation procedure**. You will only achieve generalisation **if you can measure it**

SUMMARY

What has to survive this chapter

1. The purpose of a model is to **generalise** — to be accurate on inputs it has never seen. Harder than it sounds.
2. Deep networks generalise by **interpolating** on the data's latent manifold, which is why they only make sense of inputs **close to what they have seen**.
3. The fundamental problem is the **optimisation vs generalisation tension**. Every best practice in the book manages it.
4. **Measure before you improve.** Hold-out, K-fold, iterated K-fold — and always keep a completely untouched test set.
5. **Set a common-sense baseline first.** If you cannot beat it, the problem may not be a machine learning problem.
6. To fit: **learning rate and batch size** → **architecture priors** → **capacity**, until it can overfit.
7. To generalise: **better data** → **better features** → **early stopping** → **less capacity** → **L1/L2** → **dropout**, in that order.

NOTEBOOK

02_shuffled_labels_and_manifolds.ipynb

NEXT

Chapter 6 — The universal workflow